KB금융그룹 | 국민의 평생 금융파트너

K-디지털 트레이닝

vue-router와 axios를 사용한 예제(심화 3)

[KB] IT's Your Life

KB국민은행

vue-router와 axios를 사용한 예제

- 09-2 라우팅(심화3) 문제를 기반으로 진행합니다.

vue-router와 axios를 사용한 예제

config 모듈 수정

- src/config/index.js

```
export default {
  title: 'Fake Store', // 메인 타이틀
  subtitle: 'Vue3 연습 사이트', // 서브 타이틀
  menus: [
    // 메인 메뉴 구성 정보
    {
      title: '가전',
      url: '/electronics',
      icon: 'fa-solid fa-tv',
      backgroundImage: '',
    },
    {
      title: '주얼리',
      url: '/jewelery',
      icon: 'fa-solid fa-gem',
    },
    {
      title: '남성의류',
      url: '/mensclothing',
      icon: 'fa-solid fa-user-tie',
    },
    {
      title: '여성의류',
      url: '/womensclothing',
      icon: 'fa-solid fa-person-dress',
    },
  ],
};
```

라우터 모듈 수정 src/router/index.js

```
import { createRouter, createWebHistory } from 'vue-router';

import HomePage from '../pages/HomePage.vue';

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    { path: '/', name: 'home', component: HomePage },
    {
      path: '/electronics',
      name: 'electronics',
      component: () => import('../pages/ElectronicsPage.vue'),
    },
    {
      path: '/jewelery',
      name: 'jewelery',
      component: () => import('../pages/JeweleryPage.vue'),
    },
    {
      path: '/mensclothing',
      name: "men's clothing",
      component: () => import('../pages/MensClothingPage.vue'),
    },
```

vue-router와 axios를 사용한 예제

라우터 모듈 수정 src/router/index.js

```
   {
      path: '/womensclothing',
      name: "women's clothing",
      component: () => import('../pages/WomensClothingPage.vue'),
    },
  ],
});

export default router;
```

vue-router와 axios를 사용한 예제

프록시 서버 설정을 vite.config.js에 다음과 같이 추가하세요.

```
...
export default defineConfig({
  plugins: [vue(), vueDevTools()],
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url)),
    },
  },

  server: {
    proxy: {
      '/api': {
        target: 'https://fakestoreapi.com',
        changeOrigin: true,
        rewrite: (path) => path.replace(/^\/api/, ''),
      },
    },
  },
});
```

vue-router와 axios를 사용한 예제

- fakestore 서버 api를 확인하세요.
  - https://fakestoreapi.com/

Routes

All HTTP methods are supported

GET	/products
GET	/products/1
GET	/products/categories
GET	/products/category/jewelery
GET	/carts?userId=1
GET	/products?limit=5
POST	/products
PUT	/products/1
PATCH	/products/1
DELETE	/products/1

vue-router와 axios를 사용한 예제

- 라우팅 테이블에 다음을 추가하세요.

메뉴	요청 url	name	component
-	/product/:id	product/id	ProductDetailPage
-	/product/write	product/write	ProductWritePage
-	product/edit:id	product/edit	ProductEditPage

vue-router와 axios를 사용한 예제

src/router/index.js 추가

```
...
const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    ...
    {
      path: '/product/:id',
      name: 'product/id',
      component: () => import('../pages/ProductDetailPage.vue'),
    },
    {
      path: '/product/write',
      name: 'product/write',
      component: () => import('../pages/ProductWritePage.vue'),
    },
    {
      path: '/product/edit/:id',
      name: 'product/edit',
      component: () => import('../pages/ProductEditPage.vue'),
    },
  ],
});
...
```

vue-router와 axios를 사용한 예제

- fakestore 서버와 통신하는 api 모듈을 완성하세요.

src/api/product.js

```
import axios from 'axios';

const BASE = '/api/products';

export const get = async (target, params) => {

};

export const post = async (target, product) => {

};

export const put = async (target, product) => {

};

export const remove = async (target, params) => {

};
```

src/api/product.js

```
import axios from 'axios';

const BASE = '/api/products';

export const get = async (target, params) => {
  try {
    console.log('[GET]', BASE + target, params);
    const { data } = await axios.get(BASE + target, { params });
    return data;
  } catch (e) {
    console.log('[ERROR]', e);
  }
  return [];
};

export const post = async (target, product) => {
  try {
    console.log('[POST]', BASE + target, product);
    const { data } = await axios.post(BASE + target, product);
    return data;
  } catch (e) {
    console.error(e);
  }
  return {};
};
```

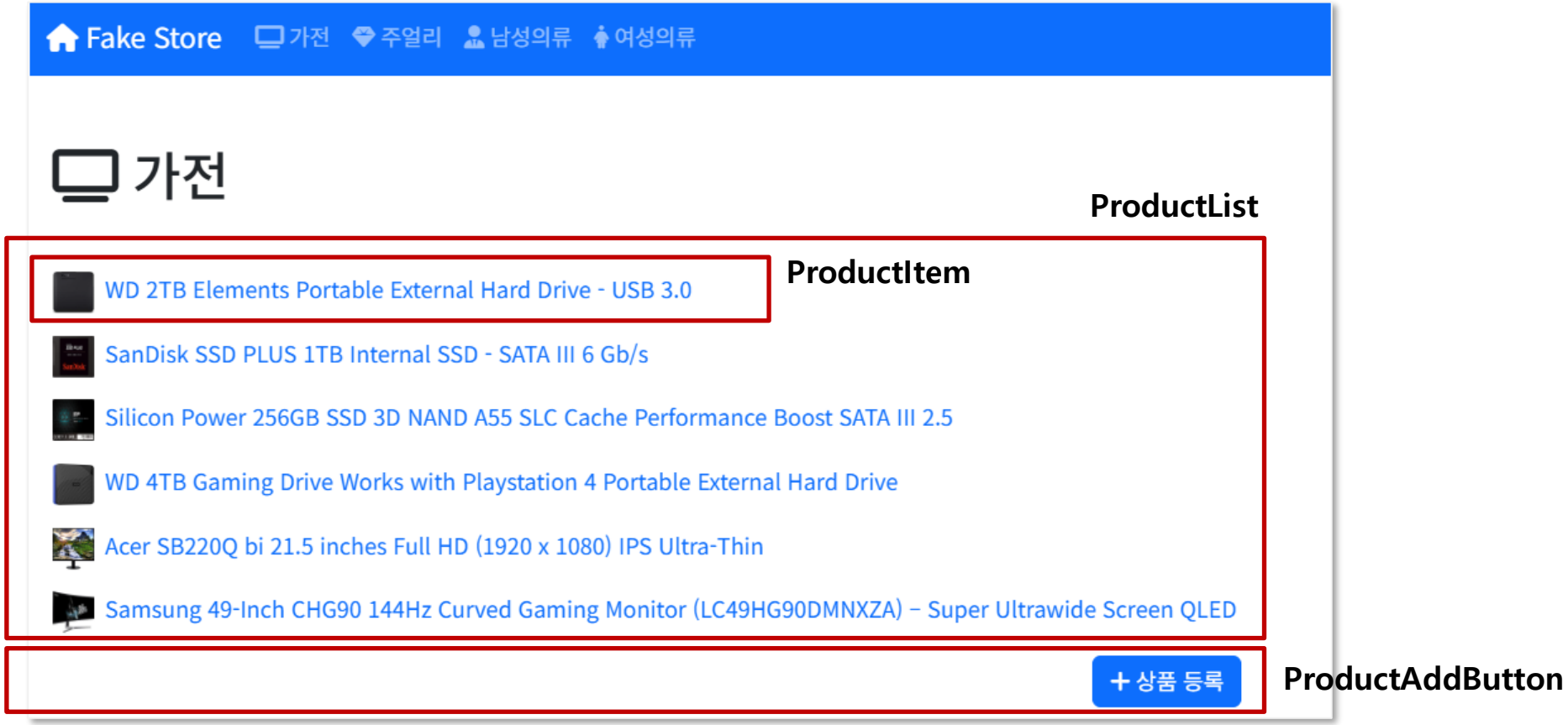
vue-router와 axios를 사용한 예제

src/api/product.js

```
export const put = async (target, product) => {
  try {
    console.log('[PUT]', BASE + target, product);
    const { data } = await axios.put(BASE + target, product);
    return data;
  } catch (e) {
    console.error(e);
  }
  return {}
};

export const remove = async (target, params) => {
  try {
    console.log('[DELETE]', BASE + target, params);
    const { data } = await axios.delete(BASE + target, { params });
    return data;
  } catch (e) {
    console.error(e);
  }
  return {};
};
```

vue-router와 axios를 사용한 예제

- 목록 보기에 사용할 기본 컴포넌트를 작성하세요.
  - components/
    - ProductItem.vue
    - ProductList.vue
    - ProductAddButton



src/components/product/ProductItem.vue

```
<script setup>
import { RouterLink } from 'vue-router';

const props = defineProps({ product: { type: Object, required: true } });
console.log(props.product);
</script>

<template>
  <div class="my-3">
    <img :src="product.image" width="30" height="30" class="me-2" />
    <RouterLink :to="{ name: 'product/id', params: { id: product.id } }">
      {{ product.title }}
    </RouterLink>
  </div>
</template>

<style scoped>
a {
  text-decoration: none;
}
</style>
```

vue-router와 axios를 사용한 예제

src/components/product/ProductList.vue

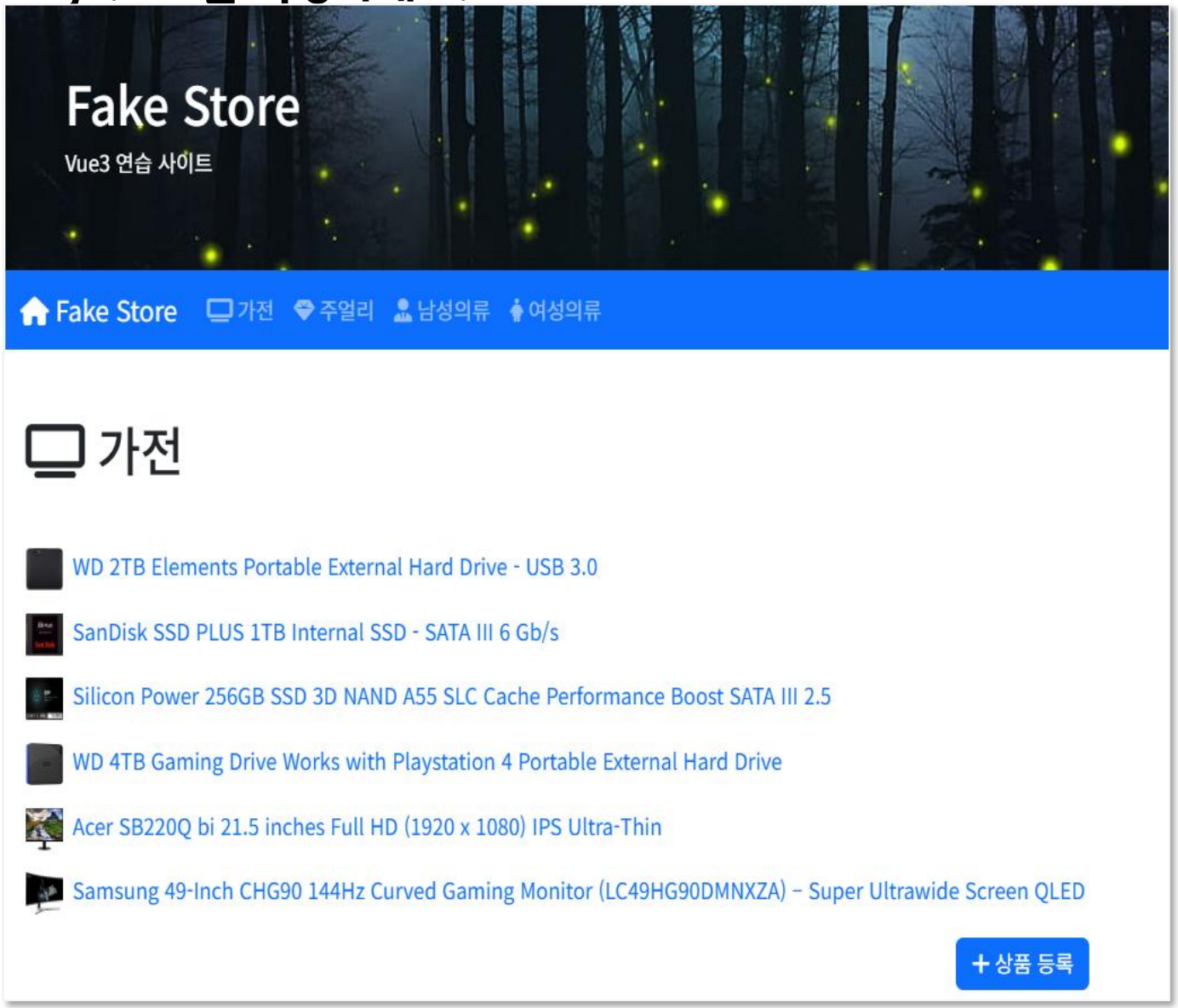
```
<script setup>
import ProductItem from './ProductItem.vue';

const props = defineProps({
  products: { type: Array, required: true },
});
</script>

<template>
  <div>
    <ProductItem v-for="product in products" :key="product.id" :product="product"/>
  </div>
</template>
```

src/components/product/ProductAddButton.vue

```
<script setup>
import { RouterLink } from 'vue-router';

const props = defineProps({
  category: { type: String, required: true },
});
</script>

<template>
  <div class="my-3 text-end me-5">
    <RouterLink :to="{ name: 'product/write', query: { category } }" class="btn btn-primary">
      <i class="fa-solid fa-plus"></i> 상품 등록
    </RouterLink>
  </div>
</template>
```

vue-router와 axios를 사용한 예제

- 가전 목록 페이지 src/pages/ElectronicsPage.vue를 작성하세요.
  - 조건
    - 상품 등록 버튼 클릭 시
      - 상품 추가 페이지로 이동
      - 카테고리 문자열을 query로 전달



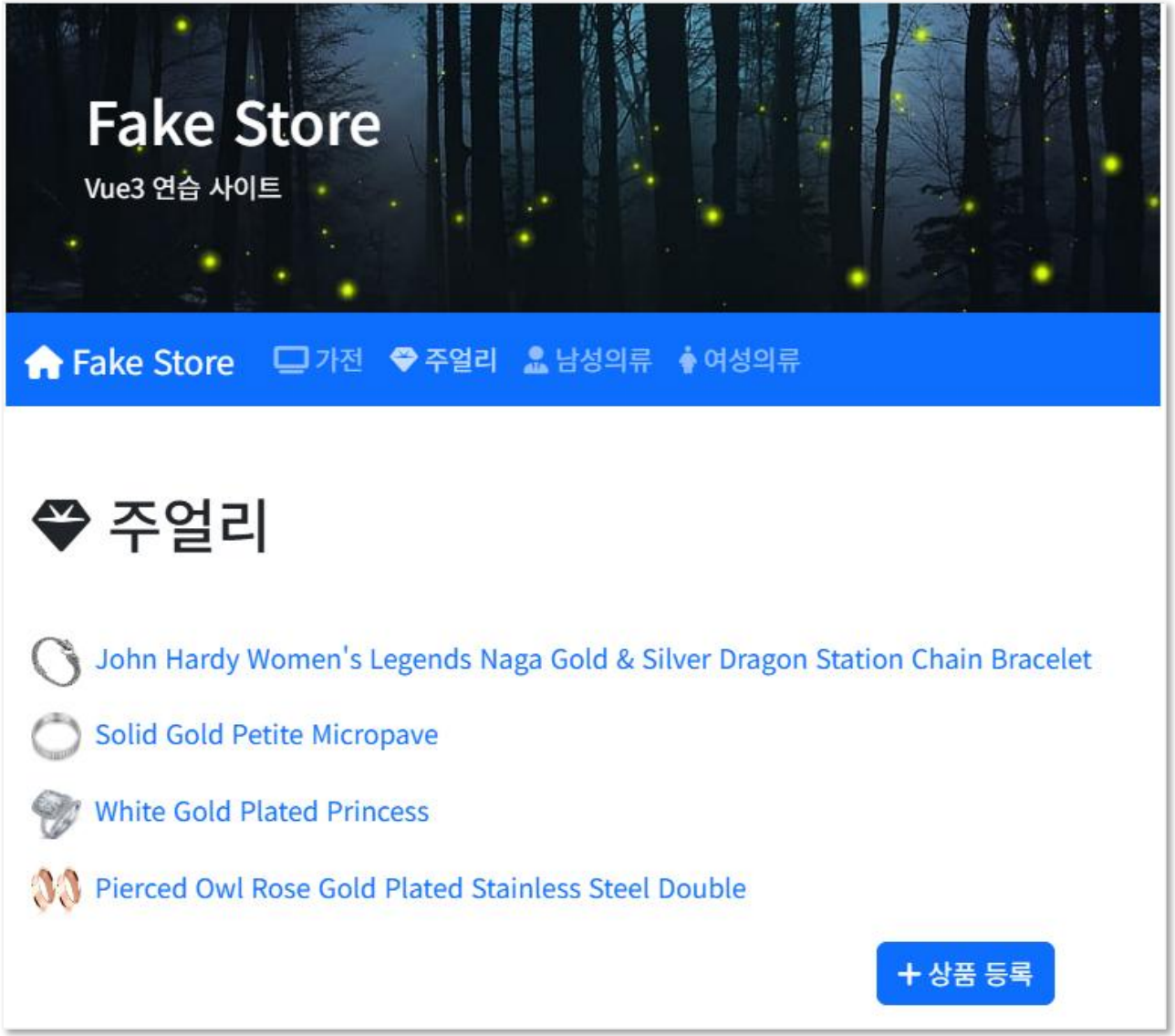
vue-router와 axios를 사용한 예제

src/pages/ElectronicsPage.vue

```
<script setup>
import * as api from '@/api/product.js';
import ProductAddButton from '@/components/product/ProductAddButton.vue';
import ProductList from '@/components/product/ProductList.vue';
import { reactive } from 'vue';

const category = 'electronics';
const state = reactive({ products: [] });

const load = async () => {
  state.products = await api.get('/category/' + category);
  console.log(state.products);
};
load();
</script>

<template>
  <h1 class="mb-5"><i class="fa-solid fa-tv"></i> 가전</h1>

  <ProductList :products="state.products" />

  <ProductAddButton :category="category" />
</template>
```

vue-router와 axios를 사용한 예제

- 주얼리 페이지 src/pages/JeweleryPage.vue를 작성하세요.
  - 조건
    - 상품 등록 버튼 클릭 시
      - 상품 추가 페이지로 이동
      - 카테고리 문자열을 query로 전달



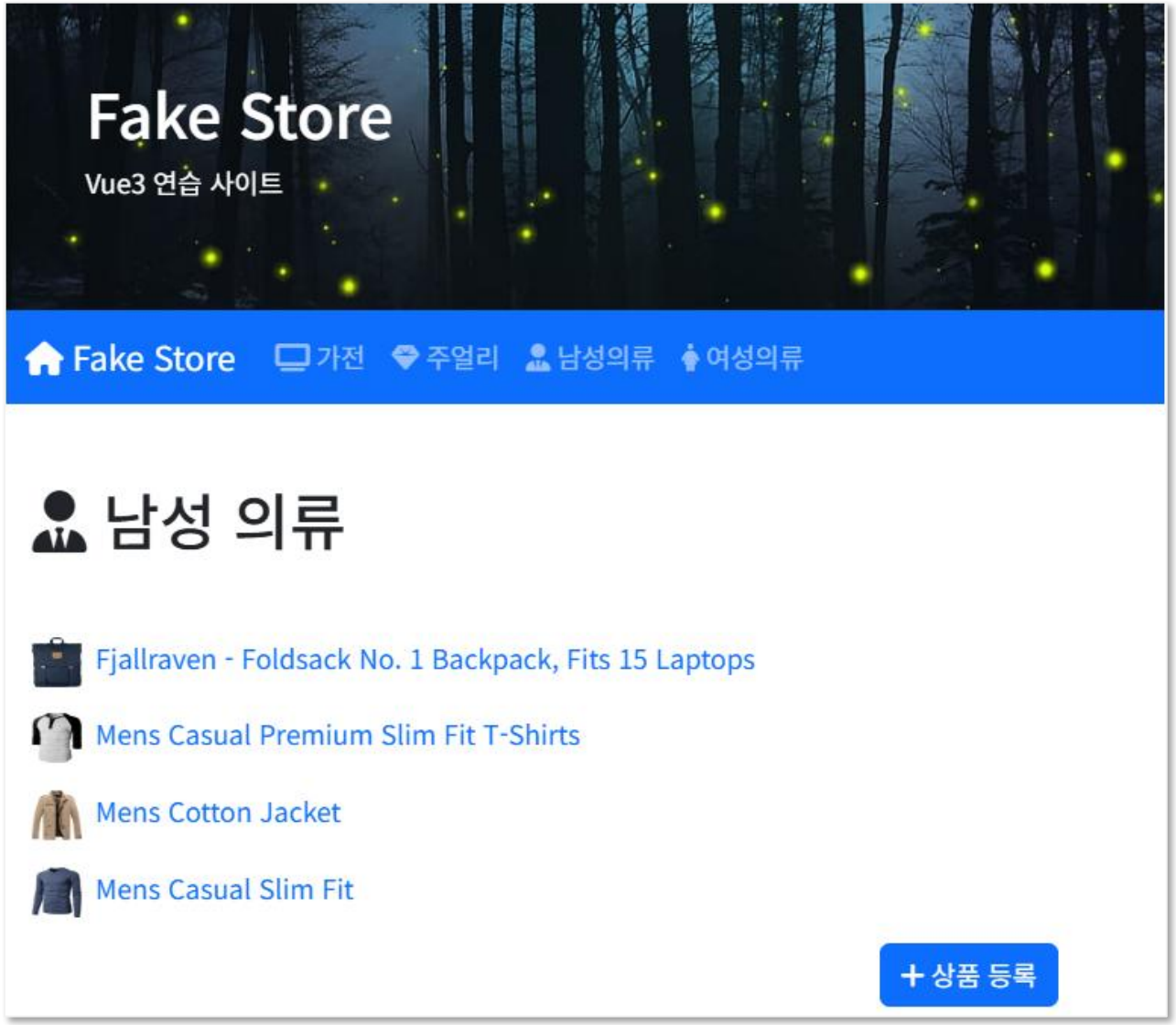
vue-router와 axios를 사용한 예제

src/pages/JeweleryPage.vue

```
<script setup>
import * as api from '@/api/product.js';
import ProductAddButton from '@/components/product/ProductAddButton.vue';
import ProductList from '@/components/product/ProductList.vue';
import { reactive } from 'vue';

const category = 'jewelery';
const state = reactive({ products: [] });

const load = async () => {
  state.products = await api.get('/category/' + category);
  console.log(state.products);
};
load();
</script>

<template>
  <h1 class="mb-5"><i class="fa-solid fa-gem"></i> 주얼리</h1>

  <ProductList :products="state.products" />

  <ProductAddButton :category="category" />
</template>
```

vue-router와 axios를 사용한 예제

- 남성의류 페이지 src/pages/MensClothingPage.vue를 작성하세요.
  - 조건
    - 상품 등록 버튼 클릭 시
      - 상품 추가 페이지로 이동
      - 카테고리 문자열을 query로 전달

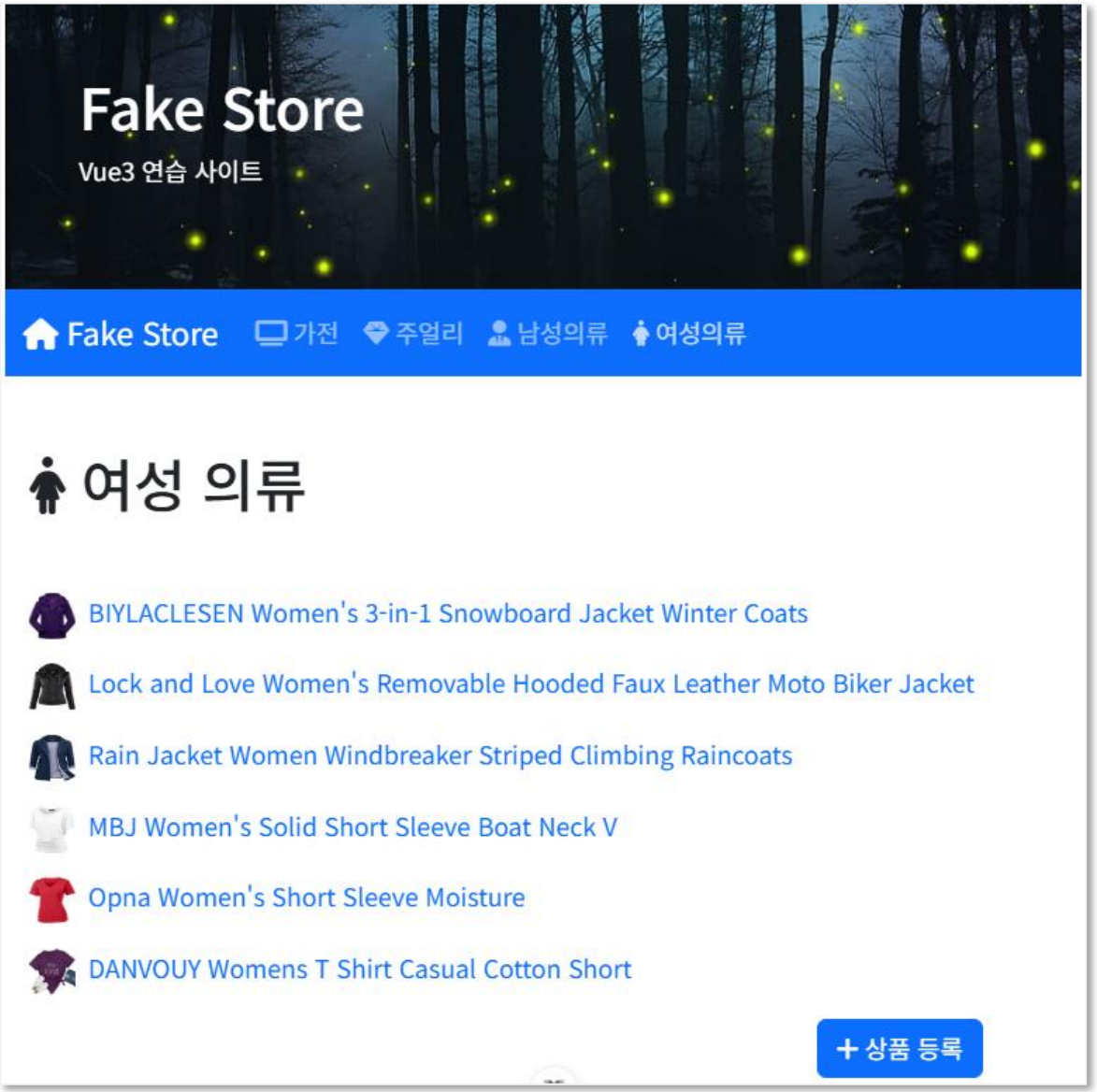


src/pages/MensClothingPage.vue

```
<script setup>
import * as api from '@/api/product.js';
import ProductAddButton from '@/components/product/ProductAddButton.vue';

import ProductList from '@/components/product/ProductList.vue';
import { reactive } from 'vue';

const category = "men's clothing";
const state = reactive({ products: [] });

const load = async () => {
  state.products = await api.get('/category/' + category);
  console.log(state.products);
};
load();
</script>

<template>
  <h1 class="mb-5"><i class="fa-solid fa-user-tie"></i> 남성 의류</h1>

  <ProductList :products="state.products" />

  <ProductAddButton :category="category" />
</template>
```

vue-router와 axios를 사용한 예제

- 여성의류 페이지 src/pages/WomensClothingPage.vue를 작성하세요.
  - 조건
    - 상품 등록 버튼 클릭 시
      - 상품 추가 페이지로 이동
      - 카테고리 문자열을 query로 전달



src/pages/WomensClothingPage.vue

```
<script setup>
import * as api from '@/api/product.js';
import ProductAddButton from '@/components/product/ProductAddButton.vue';
import ProductList from '@/components/product/ProductList.vue';
import { reactive } from 'vue';

const category = "women's clothing";
const state = reactive({ products: [] });

const load = async () => {
  state.products = await api.get('/category/' + category);
  console.log(state.products);
};
load();
</script>

<template>
  <h1 class="mb-5"><i class="fa-solid fa-person-dress"></i> 여성 의류</h1>

  <ProductList :products="state.products" />

  <ProductAddButton :category="category" />
</template>
```

vue-router와 axios를 사용한 예제

상품 추가 페이지 ProductWritePage.vue를 작성하세요.

- 조건
  - 카테고리는 목록 페이지에서 전달함
  - 상품명(title)은 필수이며 5글자 이상이여야 함
  - 확인 버튼은 title 정보가 유효해지만 활성화됨
  - 확인 버튼을 클릭하면 추가할지 물어보고 진행함
  - 추가 성공 시 해당 카테고리의 목록 페이지로 이동

vue-router와 axios를 사용한 예제

src/pages/ProductWritePage.vue

```
<script setup>
import { computed, reactive } from 'vue';
import { RouterLink, useRoute, useRouter } from 'vue-router';
import * as api from '@/api/product';

const router = useRouter();
const cr = useRoute();
const { category } = cr.query;
const product = reactive({
  category,
});

const isDisable = computed(() => !product.title || product.title.length < 5);

const submit = async () => {
  if (!confirm('등록할까요?')) return;
  const result = await api.post('/', product);
  console.log('등록결과', result);
  router.push({ name: category });
};
</script>
```

src/pages/ProductWritePage.vue

```
<template>
  <form @submit.prevent="submit" class="mx-auto" style="width: 600px">
    <h2 class="my-5">상품 등록</h2>

    <div class="my-3">
      <label> 카테고리: </label>
      <div class="form-control border-0">
        {{ product.category }}
      </div>
    </div>

    <div class="my-3">
      <label> 상품명*: </label>
      <input type="text" class="form-control" v-model="product.title" />
    </div>

    <div class="my-3">
      <label> 이미지 경로: </label>
      <input type="text" class="form-control" v-model="product.image" />
    </div>

    <div class="my-3">
      <label> 설명: </label>
      <textarea class="form-control" v-model="product.description"></textarea>
    </div>
```

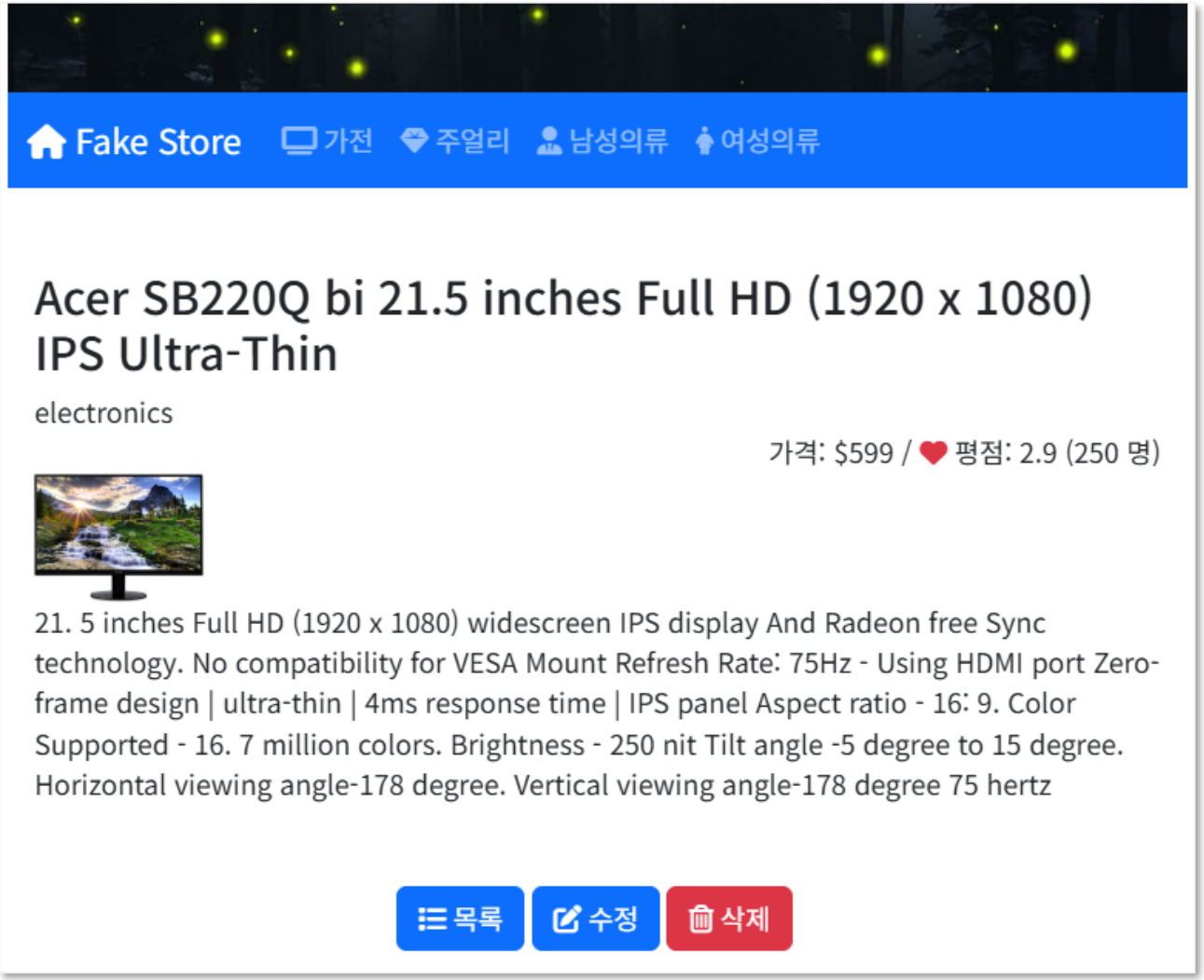
src/pages/ProductWritePage.vue

```
    <div class="text-center">
      <button class="btn btn-primary me-1" :disabled="isDisable">
        <i class="fa-solid fa-check"></i> 확인
      </button>
      <RouterLink :to="{ name: category }" class="btn btn-primary">
        <i class="fa-solid fa-undo"></i> 취소
      </RouterLink>
    </div>
  </form>
</template>
```

vue-router와 axios를 사용한 예제

상품 상세 보기 페이지 ProductDetailPage.vue를 작성하세요.

- 조건
  - 목록 버튼을 누르면 해당 카테고리의 목록 페이지로 이동
  - 수정을 누르면 수정 페이지로 이동
  - 삭제버튼을 누르면 삭제할 지 물어보고 진행
  - 삭제 성공 시 해당 카테고리의 목록 페이지로 이동



vue-router와 axios를 사용한 예제

src/pages/ProductDetailPage.vue

```
<script setup>
import { RouterLink, useRoute, useRouter } from 'vue-router';
import * as api from '@/api/product.js';
import { ref } from 'vue';

const router = useRouter();
const cr = useRoute();
const id = parseInt(cr.params.id);
const product = ref({});

console.log(id);

const remove = async () => {
  if (!confirm('삭제할까요?')) return;
  await api.remove('/' + id);
  router.push({ name: product.value.category });
};

const load = async () => {
  product.value = await api.get('/' + id);
  console.log(product.value);
};
load();
</script>
```

src/pages/ProductDetailPage.vue

```
<template>
  <h3>{{ product.title }}</h3>
  <div>{{ product.category }}</div>
  <div class="text-end">
    가격: ${{ product.price }} / <i class="fa-solid fa-heart text-danger"></i> 평점:
    {{ product.rating && product.rating.rate }} ({{ product.rating && product.rating.count }} 명)
  </div>
  <div>
    <img :src="product.image" width="100px" />
  </div>
  <div>
    {{ product.description }}
  </div>
```

src/pages/ProductDetailPage.vue

```
  <div class="text-center my-5">
    <RouterLink class="btn btn-primary me-1" :to="{ name: product.category }">
      <i class="fa-solid fa-list"></i> 목록
    </RouterLink>
    <RouterLink
      class="btn btn-primary me-1"
      :to="{ name: 'product/edit', params: { id: product.id } }"
    >
      <i class="fa-solid fa-pen-to-square"></i> 수정
    </RouterLink>
    <button class="btn btn-danger me-1" @click="remove">
      <i class="fa-regular fa-trash-can"></i> 삭제
    </button>
  </div>
</template>
```

vue-router와 axios를 사용한 예제

상품 정보 수정 페이지 ProductEditPage.vue를 작성하세요.

- 조건
  - 카테고리는 상품의 카테고리로 수정 불가
  - 상품명(title)은 필수이며 5글자 이상이여야 함
  - 확인 버튼은 title 정보가 유효해지만 활성화됨
  - 확인 버튼을 클릭하면 수정할지 물어보고 진행함
  - 수정 성공 시 해당 상품의 상세보기 페이지로 이동

vue-router와 axios를 사용한 예제

src/pages/ProductEditPage.vue

```
<script setup>
import { computed, reactive } from 'vue';
import { RouterLink, useRoute, useRouter } from 'vue-router';
import * as api from '@/api/product';

const router = useRouter();
const cr = useRoute();
const { category } = cr.query;
const id = parseInt(cr.params.id);

const product = reactive({ category });

const isDisable = computed(() => !product.title || product.title.length < 5);

const submit = async () => {
  if (!confirm('수정할까요?')) return;
  const result = await api.put('/' + id, product);
  console.log('수정 결과:', result);
  router.push({ name: 'product/id', params: { id: product.id } });
};
```

vue-router와 axios를 사용한 예제

src/pages/ProductEditPage.vue

```
const load = async () => {
  const data = await api.get('/' + id);
  Object.assign(product, data);

  console.log(product);
};
load();
</script>

<template>
  <form @submit.prevent="submit" class="mx-auto" style="width: 600px">
    <h2 class="my-5">상품 수정</h2>

    <div class="my-3">
      <label> 카테고리: </label>
      <div class="form-control border-0">
        {{ product.category }}
      </div>
    </div>

    <div class="my-3">
      <label> 상품명*: </label>
      <input type="text" class="form-control" v-model="product.title" />
    </div>
```

src/pages/ProductEditPage.vue

```
    <div class="my-3">
      <label> 이미지 경로: </label>
      <input type="text" class="form-control" v-model="product.image" />
    </div>

    <div class="my-3">
      <label> 설명: </label>
      <textarea class="form-control" v-model="product.description"></textarea>
    </div>

    <div class="text-center">
      <button class="btn btn-primary me-1" :disabled="isDisable">
        <i class="fa-solid fa-check"></i> 확인
      </button>
      <RouterLink :to="{ name: 'product/id', params: { id: product.id } }" class="btn btn-primary">
        <i class="fa-solid fa-undo"></i> 취소
      </RouterLink>
    </div>
  </form>
</template>
```